

UTA Smart Kiosk

Operations & Development Manual

The Kiosk Guys

Fall 2025 - Spring 2026

Contents

1	User Guide	2
1.1	Login Dashboard	2
1.2	Main Dashboard	3
1.3	Admin’s Dashboard	8
2	Administrator Guide	10
2.1	System Bootup & Kiosk Mode	11
2.2	Emergency Exit: Killing the Terminal	11
3	Developer & Programmer Guide	12
3.1	System Architecture & Local Execution	12
3.2	The Boot Sequence & Network Handshake	12
3.3	Critical Configuration File Paths	12
3.4	Proprietary Installation Steps & Dependencies	13
3.5	First-Time Deployment & Troubleshooting	13
3.6	Database Schema & Data Retention	14
4	API Guide	15
4.1	MappedIn	15
5	External Services & Credentials	16
6	Contact	16

1 User Guide

This section will cover how to use all features of the Smart Kiosk System through visuals.

1.1 Login Dashboard

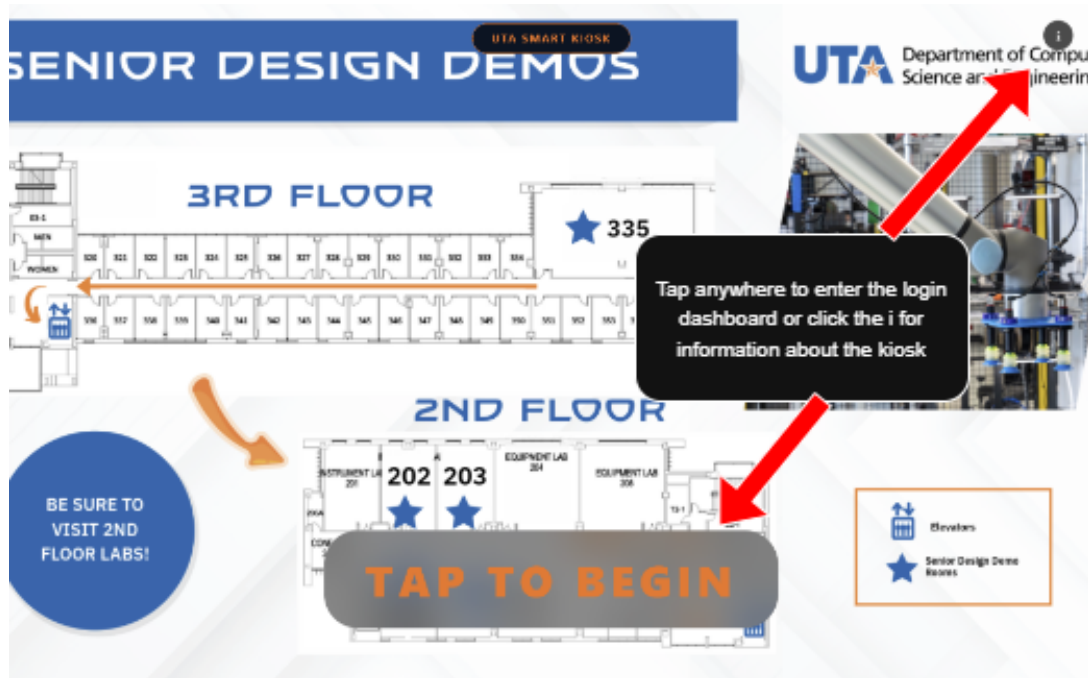


Figure 1: Idle Screen

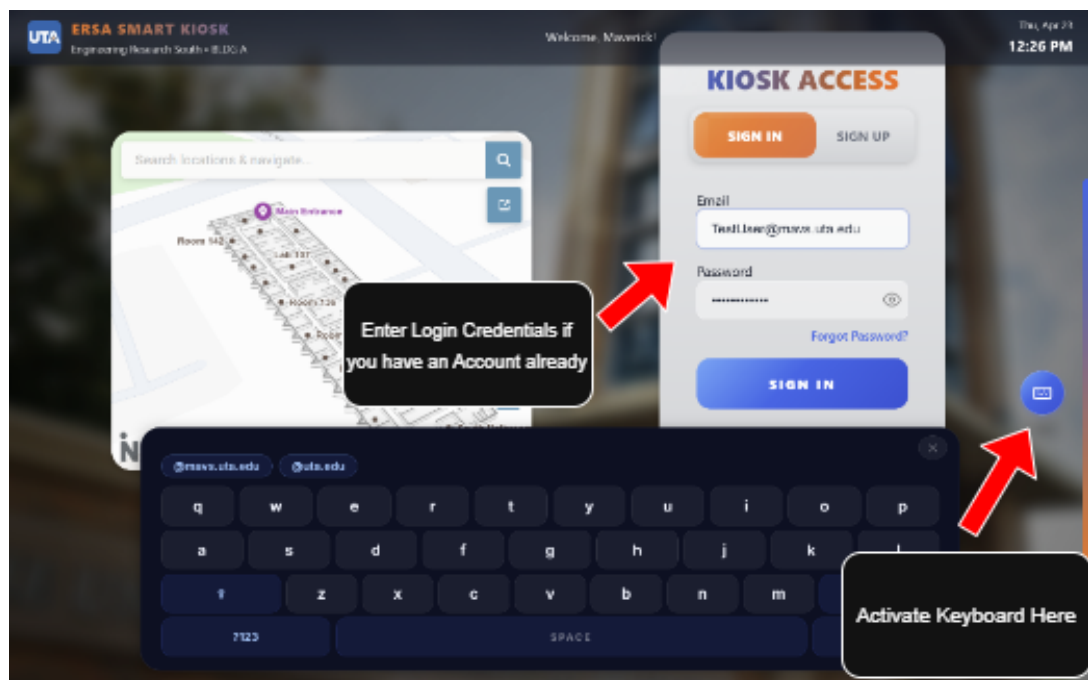


Figure 2: User Login Screen

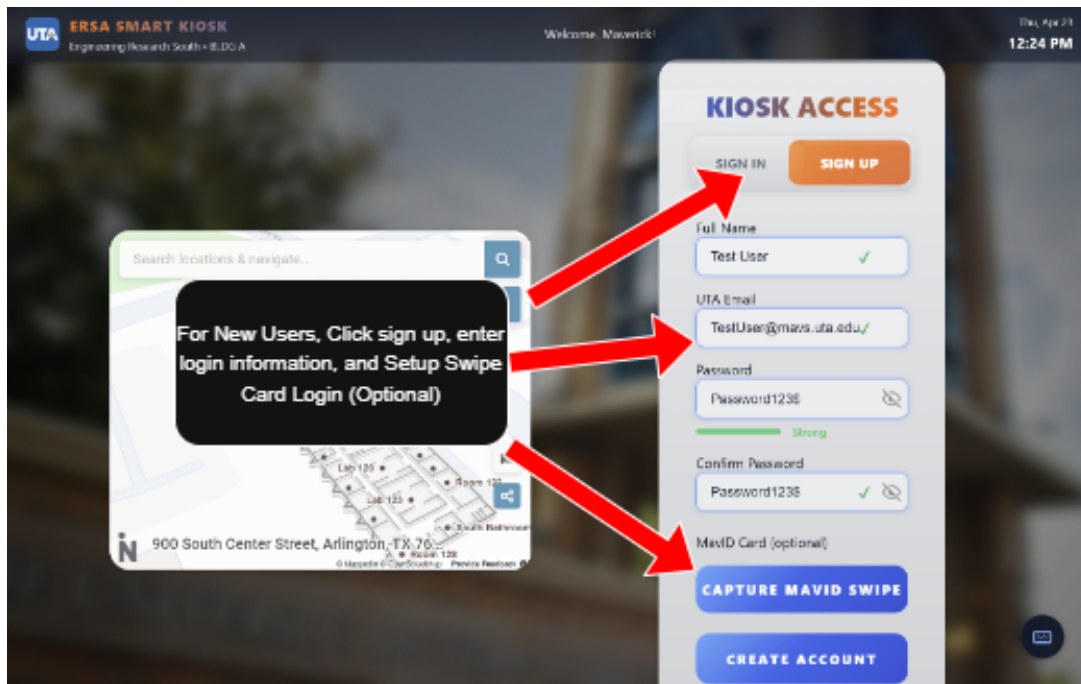


Figure 3: User Sign Up Page

1.2 Main Dashboard

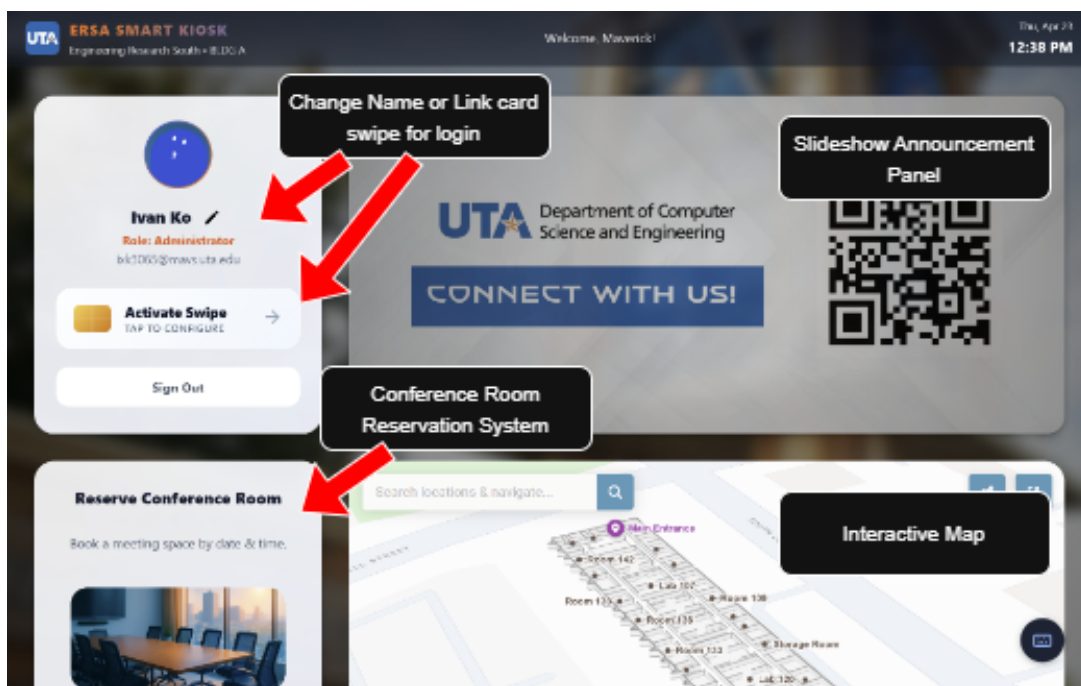


Figure 4: Main Dashboard

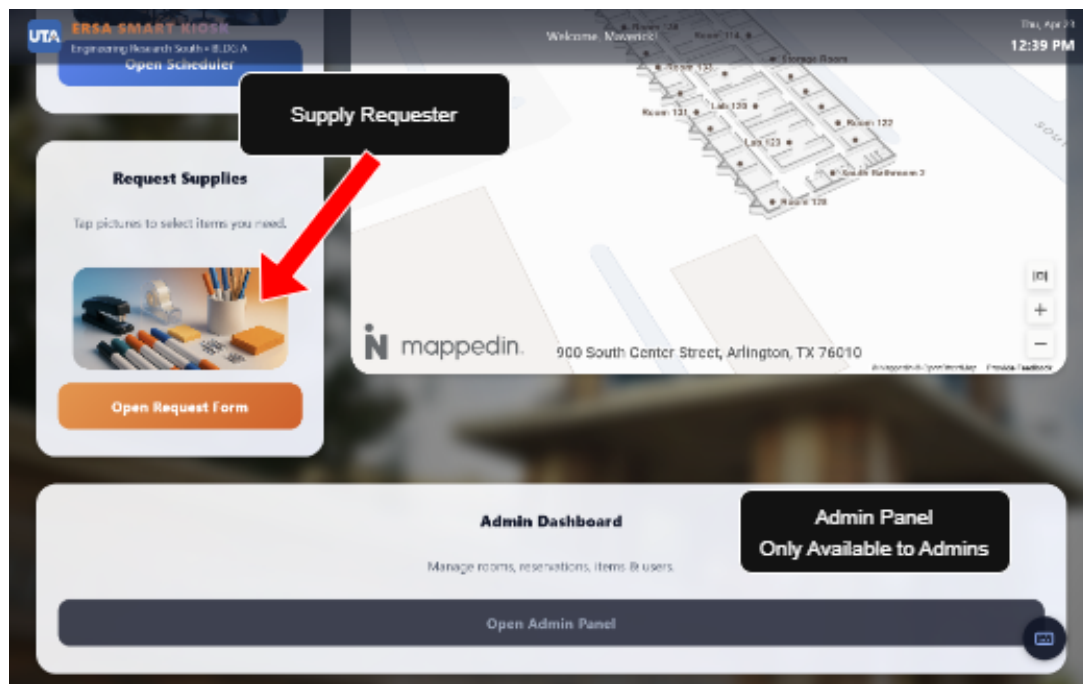


Figure 5: Main Dashboard

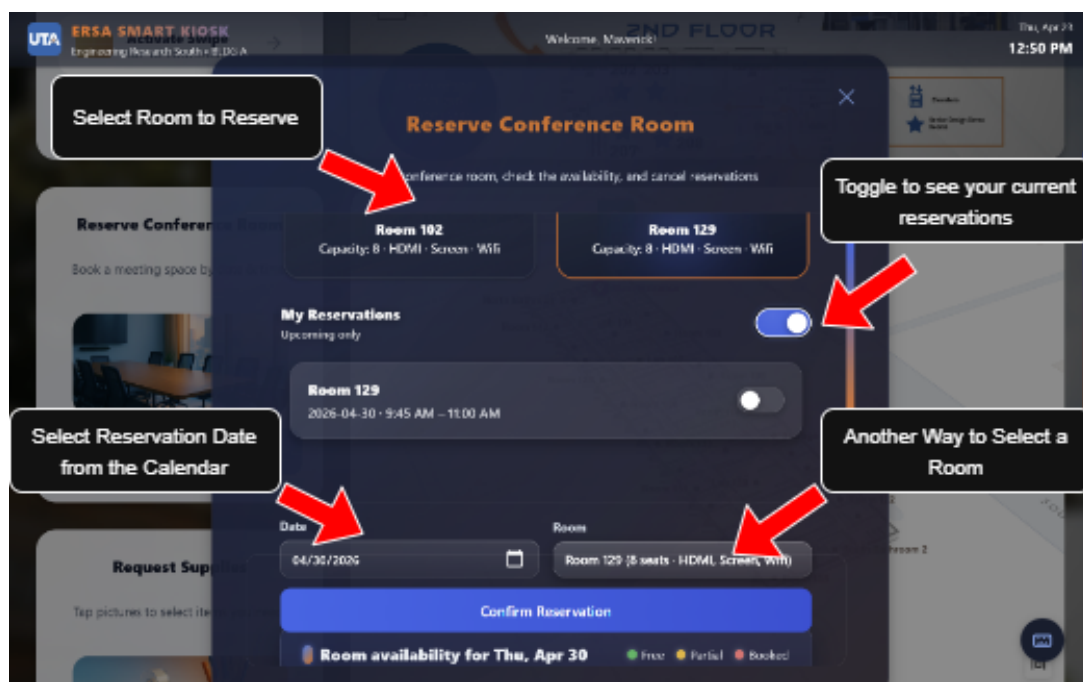


Figure 6: Conference Room Reservation

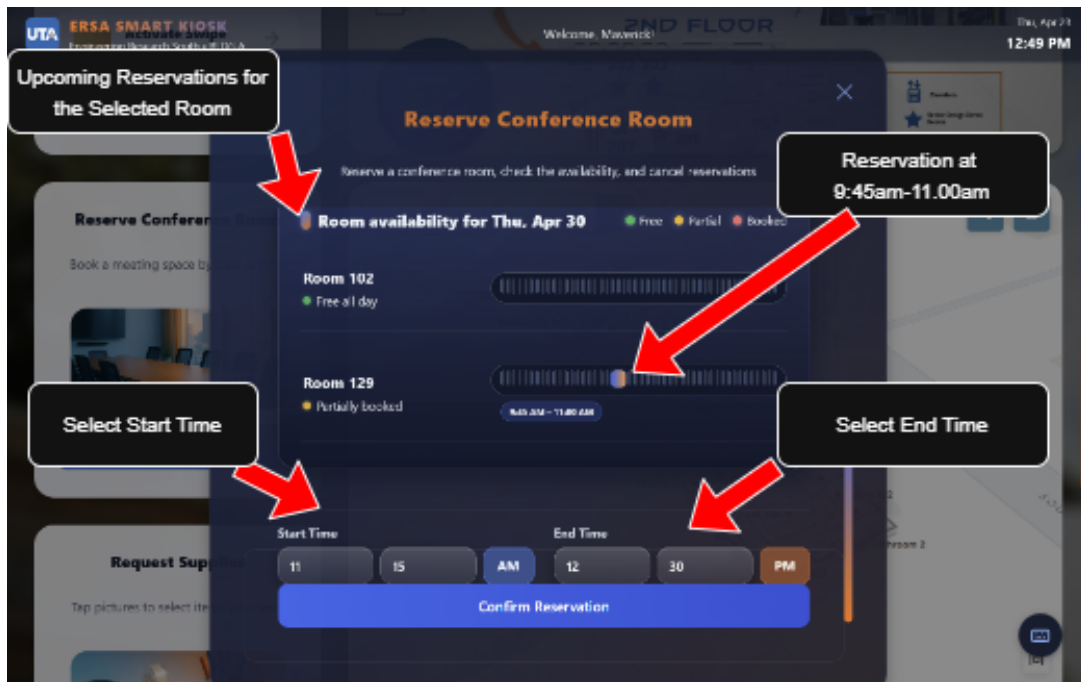


Figure 7: Conference Room Reservation

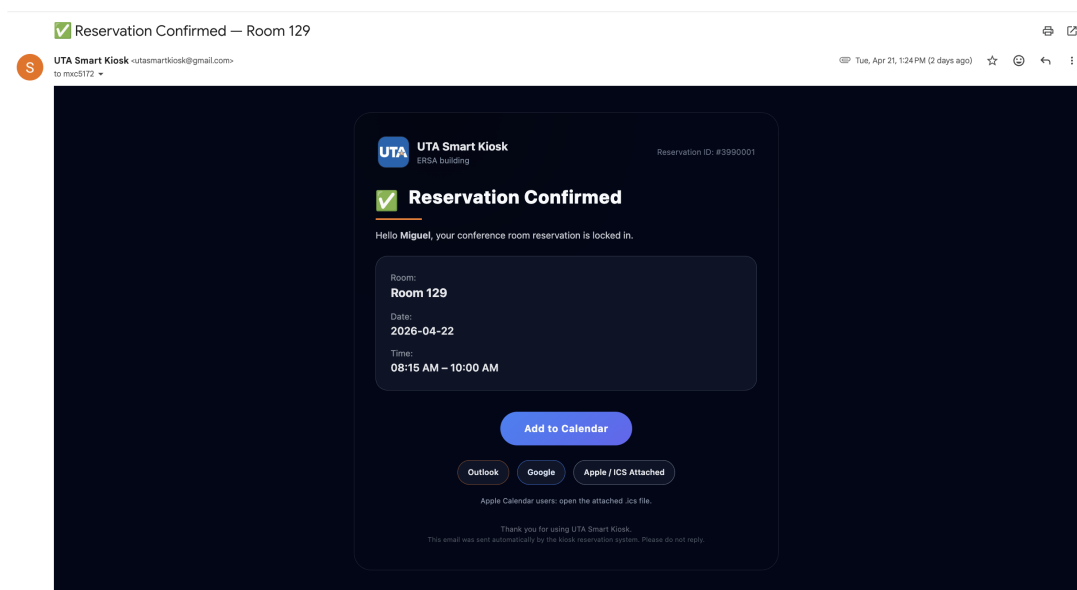


Figure 8: Room Reservation Confirmation Email

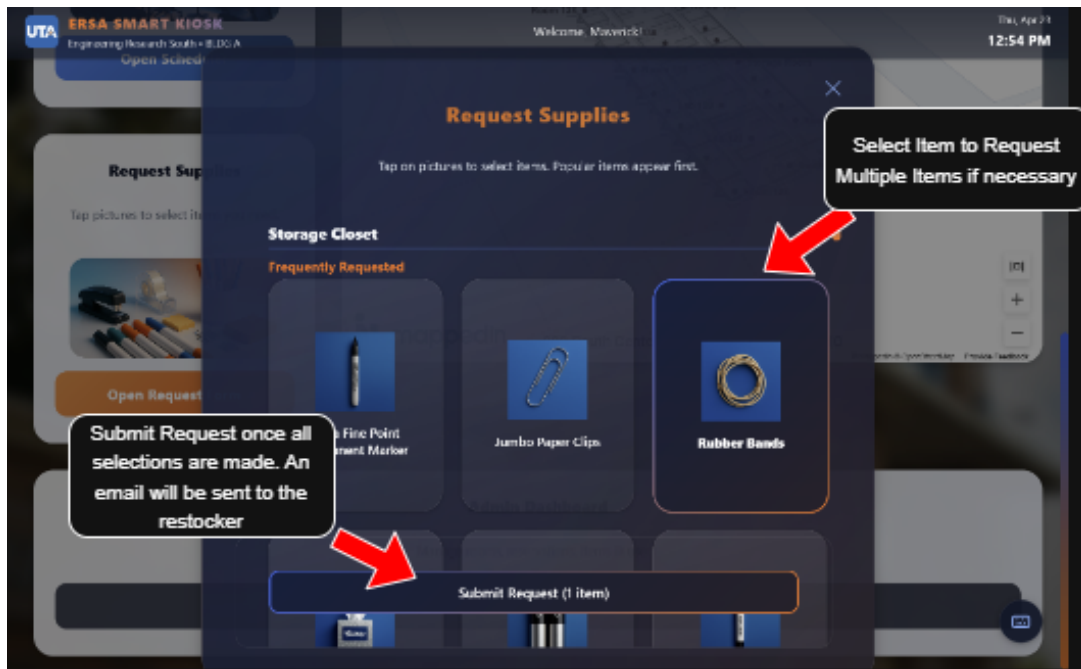


Figure 9: Supply Requester

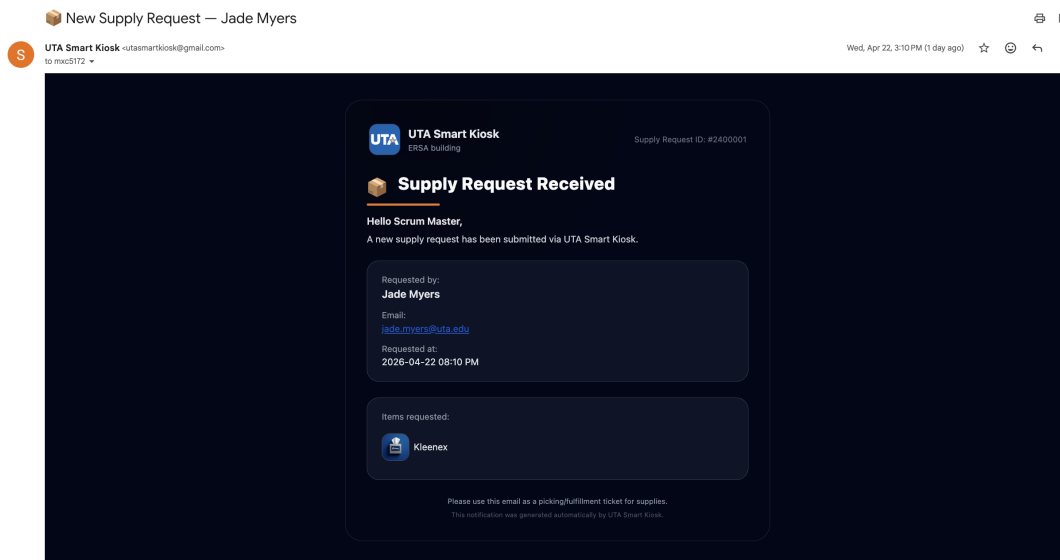


Figure 10: Supply Request Confirmation Email

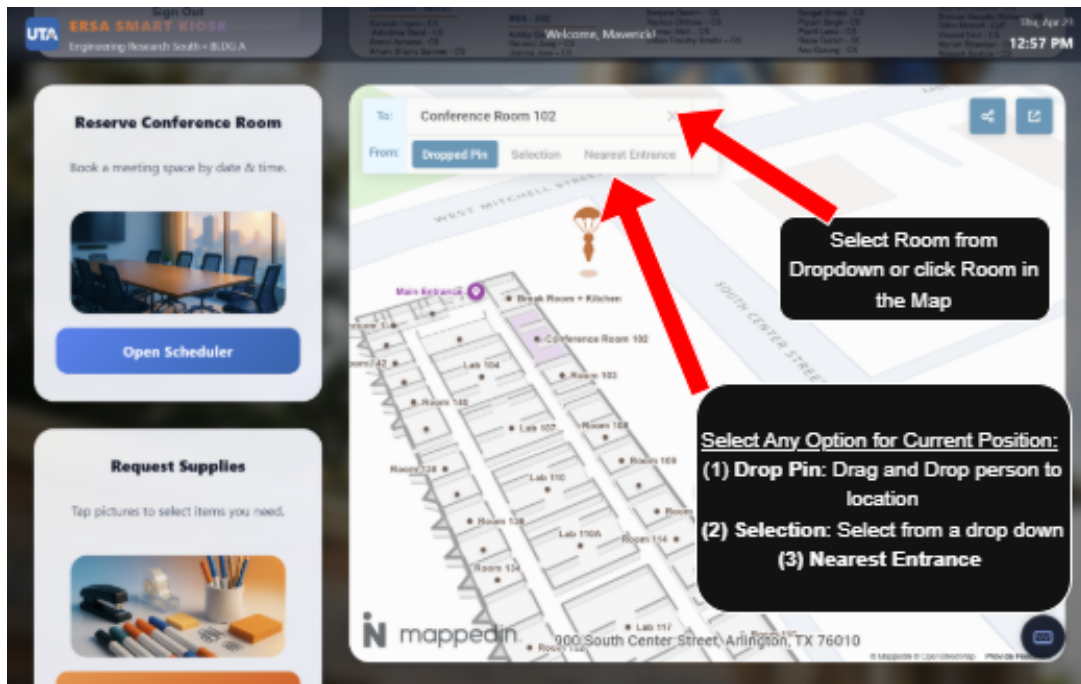


Figure 11: Interactive Map

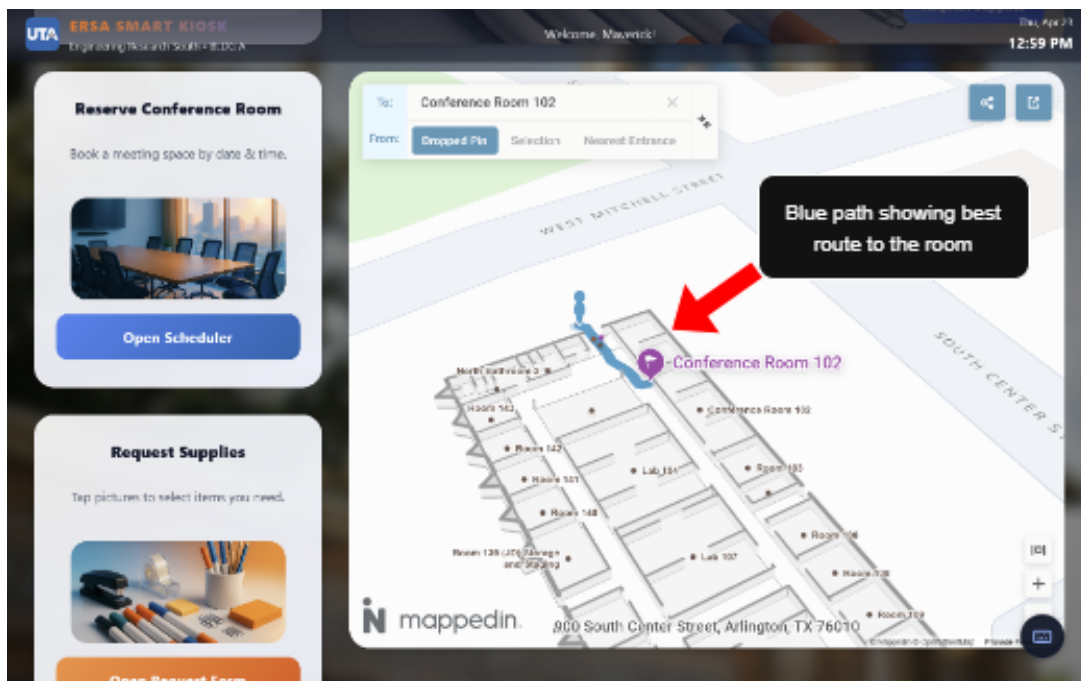


Figure 12: Interactive Map

1.3 Admin's Dashboard

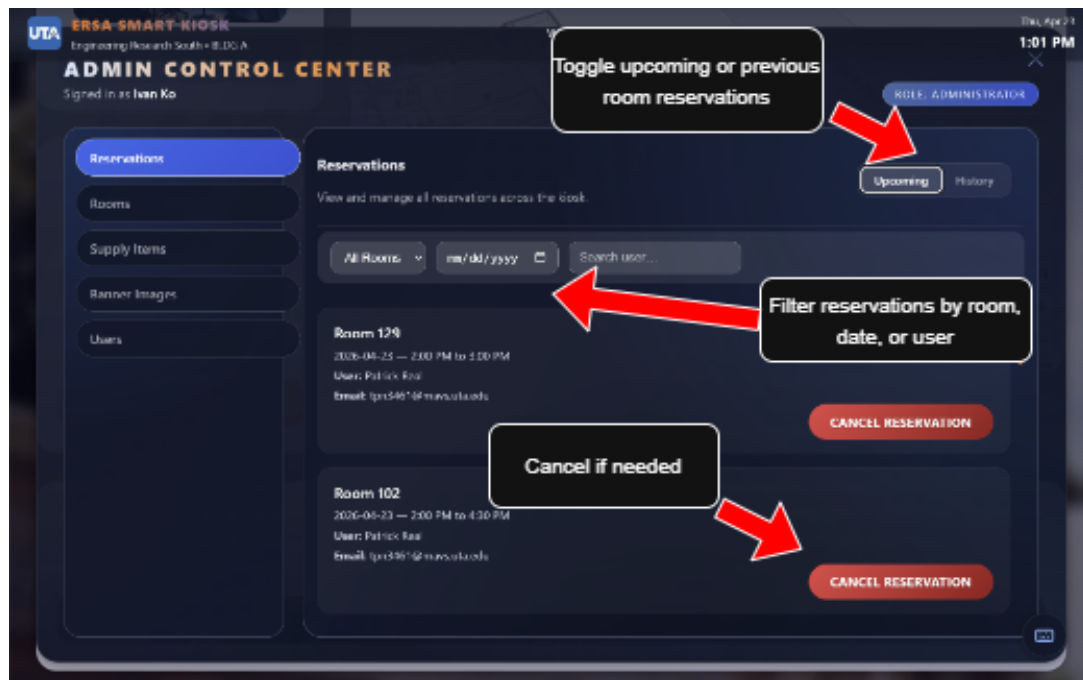


Figure 13: Room Reservation History

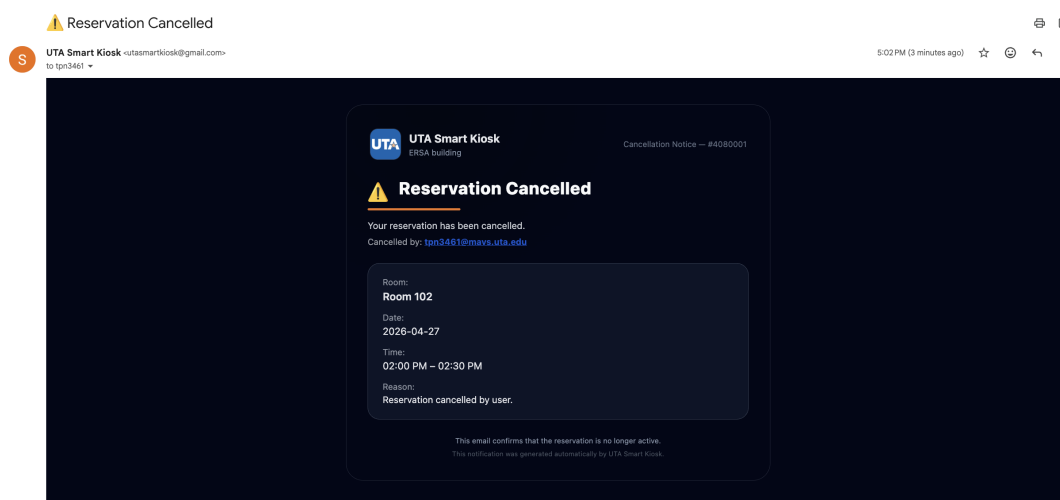


Figure 14: Cancel Room Reservation Email

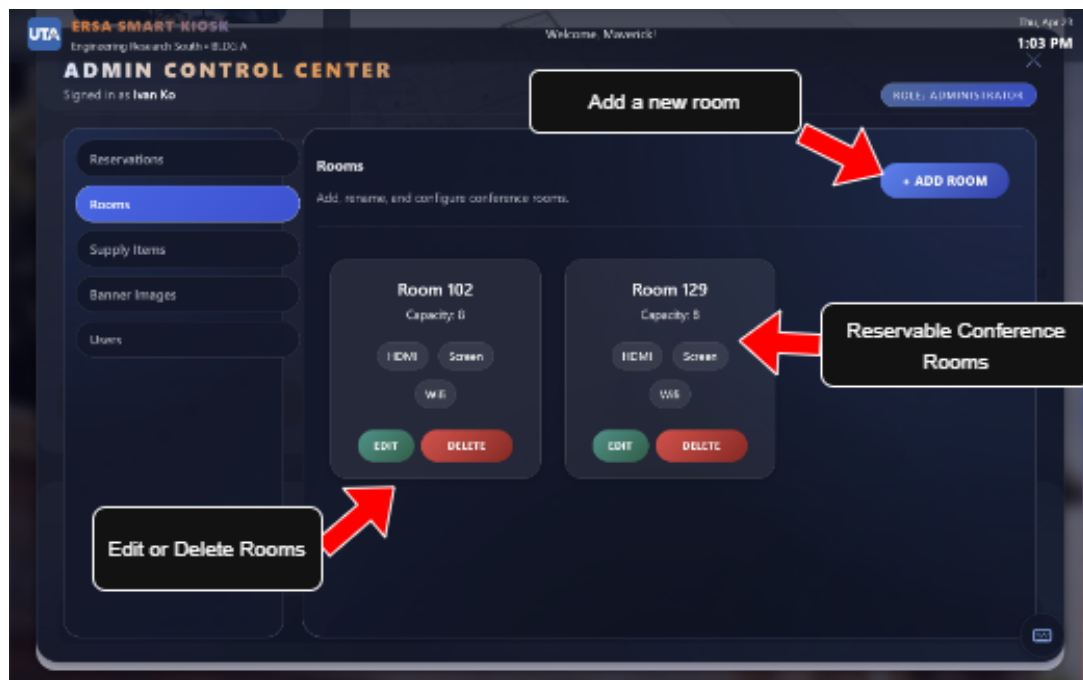


Figure 15: Edit Available Rooms

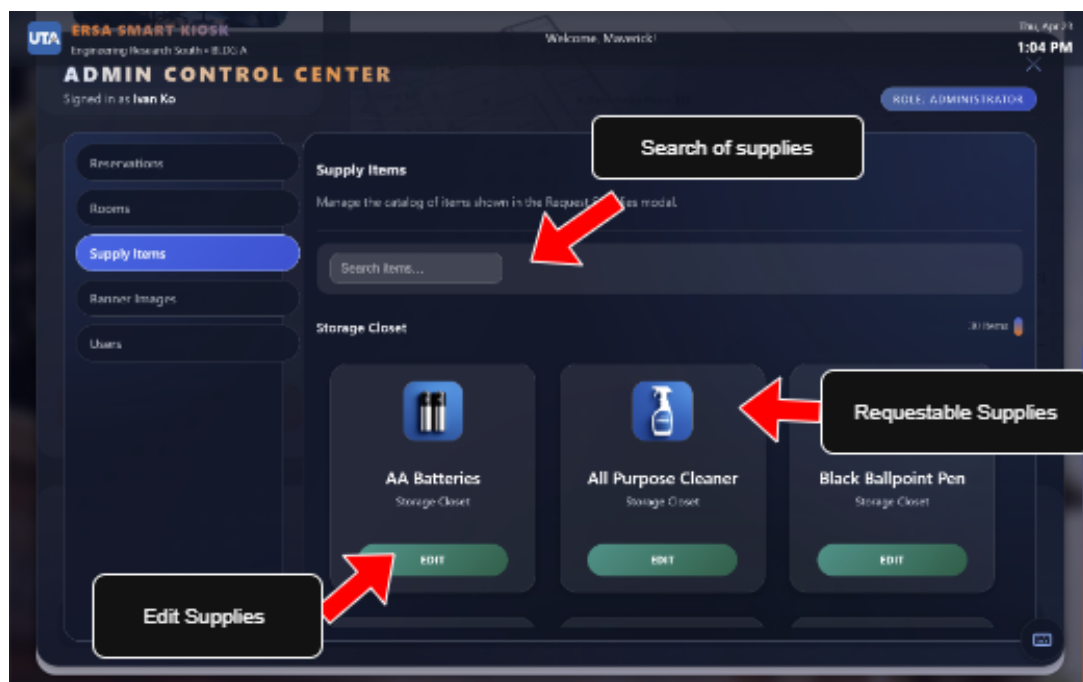


Figure 16: Edit Available Supplies

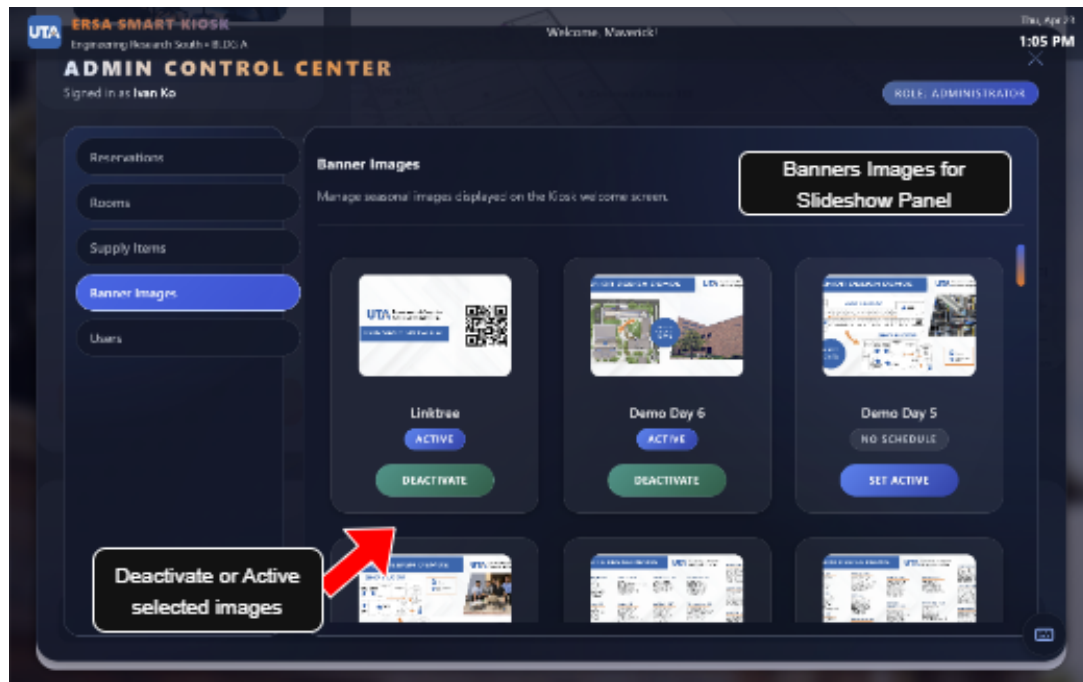


Figure 17: Edit Banner Images

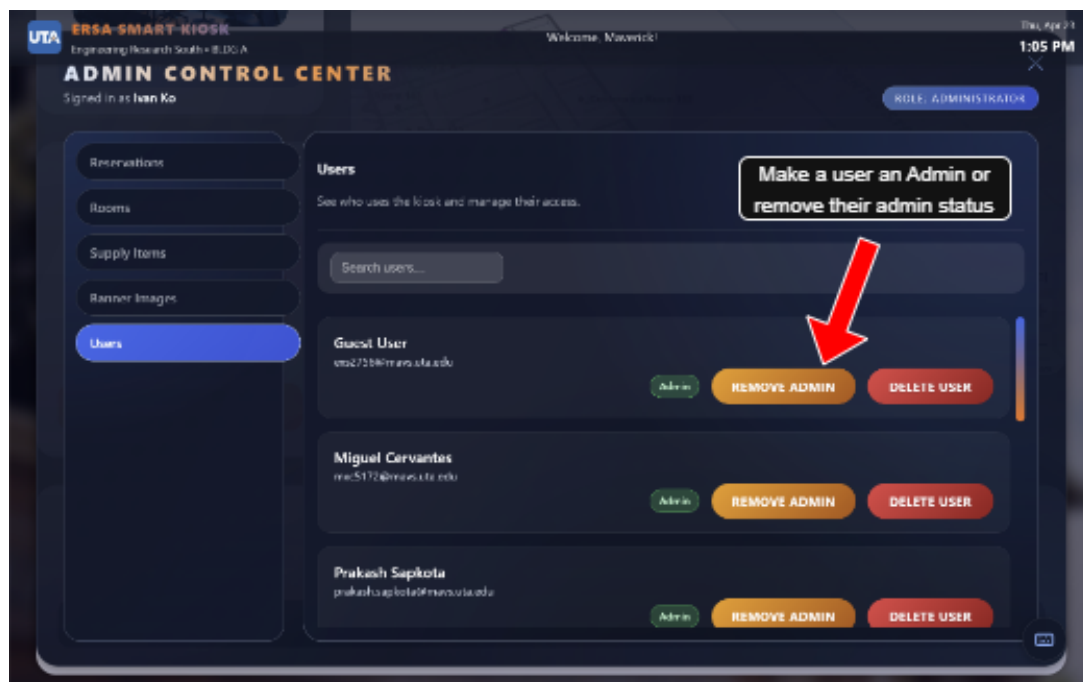


Figure 18: Edit User Access

2 Administrator Guide

This section contains standard operating procedures for administrators and staff managing the physical kiosks. No programming knowledge is required.

2.1 System Bootup & Kiosk Mode

The Smart Kiosk is designed to be completely autonomous. Upon plugging the device into a power source, the Raspberry Pi will automatically boot the operating system.

During the boot sequence, the system will seamlessly connect to the network, start all necessary background servers, and launch the user interface. Once the boot process is complete, the screen is locked into **Kiosk Mode**. This is a secure, full-screen environment that hides the underlying desktop, prevents users from closing the application, and restricts access to standard computer functions to ensure the kiosk remains dedicated to its intended purpose.

2.2 Emergency Exit: Killing the Terminal

If the kiosk becomes unresponsive, requires an emergency update, or you need access to the underlying desktop environment, follow these steps to bypass Kiosk Mode:

1. **Connect a Keyboard:** Plug a standard USB keyboard into one of the available ports on the Raspberry Pi.
2. **Access the Black-out Terminal:** Press **Ctrl + Alt + F2** simultaneously. This will minimize the graphical interface and open a black, text-only login screen.
3. **Log In:** Input the system credentials when prompted (note that the password will remain invisible as you type):

ERSA:

```
ersakiosk login: ersakiosk  
Password: KioskGuys
```

ERB:

```
ersakiosk login: erbkiosk  
Password: KioskGuys
```

4. **Stop the Kiosk Software:** Type the following exact commands, pressing Enter after each one, to terminate the locked browser and the background servers:

```
killall -9 chromium  
sudo systemctl stop smartkiosk.service
```

5. **Return to the Desktop:** Press **Alt + F7** to return to the standard graphical desktop environment. You now have full access to the machine.

3 Developer & Programmer Guide

This section details the system architecture, network configurations, and proprietary scripts required to maintain, update, and deploy the distributed three-kiosk system.

3.1 System Architecture & Local Execution

The kiosk operates entirely locally on the Raspberry Pi hardware. The source code repository is cloned directly onto the device's storage. Rather than relying on external cloud hosting for the application logic, the system utilizes a local Vite frontend, a local Django backend, and a local Python-based keystroke server.

When you pull remote updates from the GitHub repository down to the Pi, Linux security policies often strip execution privileges from shell scripts. After any `git pull` that modifies the boot logic, you **must** reapply execution permissions to both start scripts to prevent boot failures:

```
chmod +x /home/<path to script>/rpistart.sh
chmod +x /home/<path to script>/kiosk_ui.sh
```

3.2 The Boot Sequence & Network Handshake

Because the campus uses a captive portal for guest Wi-Fi, the system cannot natively connect to the internet on boot. The `rpistart.sh` script handles this by executing an automated, headless network handshake.

1. **MAC Extraction:** The script dynamically reads the Pi's internal `wlan0` MAC address.
2. **cURL Payload:** It fires a `curl` POST request containing the MAC address and specific WLAN IDs directly to the network's authentication portal.
3. **Server Initialization:** Once the handshake bypasses the captive portal, the script spins up the Django backend (port 8000), Vite frontend (port 5173), and the key-server (port 5174) in the background.

3.3 Critical Configuration File Paths

Prerequisite: Directory Creation

On a fresh Raspberry Pi OS installation, the default user profile may not have generated the configuration folders yet. Before attempting to create or edit the autostart files below, you must build the directory trees by running:

```
mkdir -p ~/.config/lxsession/LXDE-pi/
mkdir -p ~/.config/autostart/
```

The system relies on specific operating system hooks to launch the software at the correct time during the boot sequence:

- `~/.config/lxsession/LXDE-pi/autostart`
Purpose: The legacy X11 display server hook. This file executes the master

`rpistart.sh` script invisibly in the background the moment the graphical desktop environment loads.

- `~/.config/autostart/kiosk.desktop`
Purpose: The standard Linux graphical autostart entry. This runs `kiosk-ui.sh`, which waits for the background servers to confirm they are online before launching the full-screen Chromium window and the terminal splash screen.
- `/etc/systemd/system/smarkiosk.service`
Purpose: The background daemon configuration. This file registers the kiosk's backend processes with Linux `systemd`, allowing them to be managed, started, and stopped securely via standard system commands.

3.4 Proprietary Installation Steps & Dependencies

When configuring a fresh Raspberry Pi from scratch, several specific Linux packages and system commands are required beyond the standard Node and Python environments.

1. UI Management & Touchscreen Drivers

To allow the system to programmatically force windows to the top of the display hierarchy (`xdotool`), dynamically manage the touchscreen during the boot sequence (`xinput`), and automatically hide the idle mouse cursor for a clean touch experience (`unclutter`), install the following tools:

```
sudo apt update
sudo apt install xdotool xinput unclutter
```

2. Registering the Systemd Service

If modifications are made to `smarkiosk.service`, or if it is being deployed on a new machine, you must register it with the operating system before it can be utilized:

```
sudo systemctl daemon-reload
sudo systemctl enable smarkiosk.service
sudo systemctl start smarkiosk.service
```

3. Debugging the Systemd Service

Because `systemd` runs invisibly in the background, any crashes or missing dependencies will fail silently. Use the `journalctl` command to read the exact output and error logs:

```
sudo journalctl -u smarkiosk.service -e
```

3.5 First-Time Deployment & Troubleshooting

When deploying the kiosk software to a new Raspberry Pi hardware unit, you will likely encounter several environment-specific errors. Follow these steps to resolve common first-time setup issues.

1. The "Vite Not Found" Infinite Loading Loop

If the terminal splash screen gets stuck infinitely printing dots (`. . . . .`), Vite is failing to start. This happens because the Node modules have not been downloaded to the new Pi. To fix this, open `rpistart.sh`, uncomment the following line, and run the script once:

```
npm install --legacy-peer-deps
```

Critical Note: Once the installation finishes, you **must** re-comment this line out. If left active, the boot sequence will completely freeze if the Pi ever reboots without an active internet connection.

2. The NVM Path Trap

Modern Raspberry Pi OS versions frequently install the Node Version Manager to `~/.config/nvm`. However, legacy bash scripts expect it to be in `~/.nvm`. If the boot script throws an `nvm not found` error, physically move the directory and update your terminal's memory:

```
mv ~/.config/nvm ~/.nvm
nano ~/.bashrc
```

Inside `.bashrc`, scroll to the bottom and change the `NVM_DIR` export path to specifically say `"$HOME/.nvm"`. Save the file and run `source ~/.bashrc`.

3. Missing Database Client (pymysql)

If the Django backend crashes silently, it is often due to missing Python database connection wrappers. Ensure the proper MySQL client is installed inside your active virtual environment:

```
source /home/<username>/kioskguys/smartKiosk/venv/bin/
  activate
pip install pymysql
```

4. The "Untouchable" Boot Sequence

To prevent users from interacting with the operating system during bootup, the `rpistart.sh` script uses `xinput` to completely sever the touchscreen connection. It is dynamically re-enabled in `kiosk_ui.sh` the millisecond before the Chromium browser takes over the screen. If you need to manually test the UI script and find the touchscreen is dead, run `xinput list` to find your device ID, and `xinput enable <id>` to wake it back up.

3.6 Database Schema & Data Retention

The complete structural blueprint for the TiDB database is included in the attached `SmartKioskSchema.zip` file. This archive contains the raw SQL schema files detailing the tables and relationships used by the Django backend.

To ensure the database does not grow indefinitely and to automatically manage data privacy, TiDB's native Time-To-Live (TTL) feature has been enabled on several high-volume tables. This means the database will automatically sweep and delete outdated records without requiring external scripts or cron jobs.

The following TTL commands have been executed on the production database to enforce these auto-delete retention policies:

```
ALTER TABLE reservations
TTL = 'date' + INTERVAL 7 MONTH;

ALTER TABLE api_supplyrequests
```

```
TTL = 'created_at' + INTERVAL 1 MONTH;

ALTER TABLE api_passwordresetcode
TTL = 'created_at' + INTERVAL 3 DAY;
```

4 API Guide

4.1 MappedIn

The ERSA and ERB Smart Kiosks feature an interactive map service powered by MappedIn, providing a streamlined navigation experience. To update the building map, access the map editor on the MappedIn website. The bottom toolbar includes options to add structural elements such as walls, doors, stairs, and elevators. Simply select an item and drag it to the desired location on the map. Items can be removed using the undo button located on the right side of the toolbar. Selecting a room opens a panel where you can edit details such as the room name, description, and other attributes. You can preview your changes using the green button on the top right menu. All changes are applied once the editing session is closed, so no additional steps are required to publish updates. Since this service is provided by MappedIn, we are not able to address issues related to map connectivity.

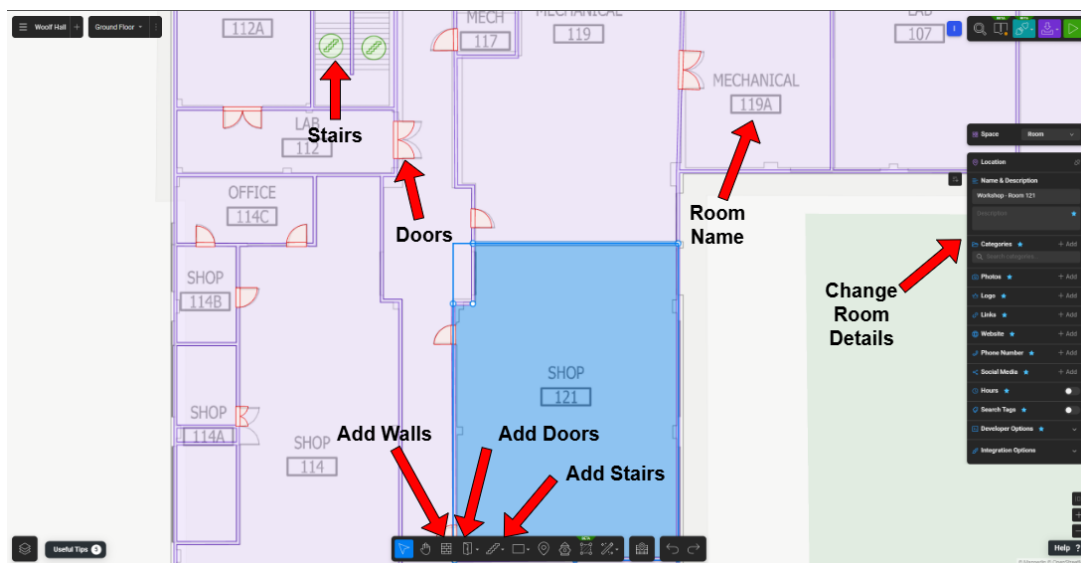


Figure 19: MappedIn Editor Guide

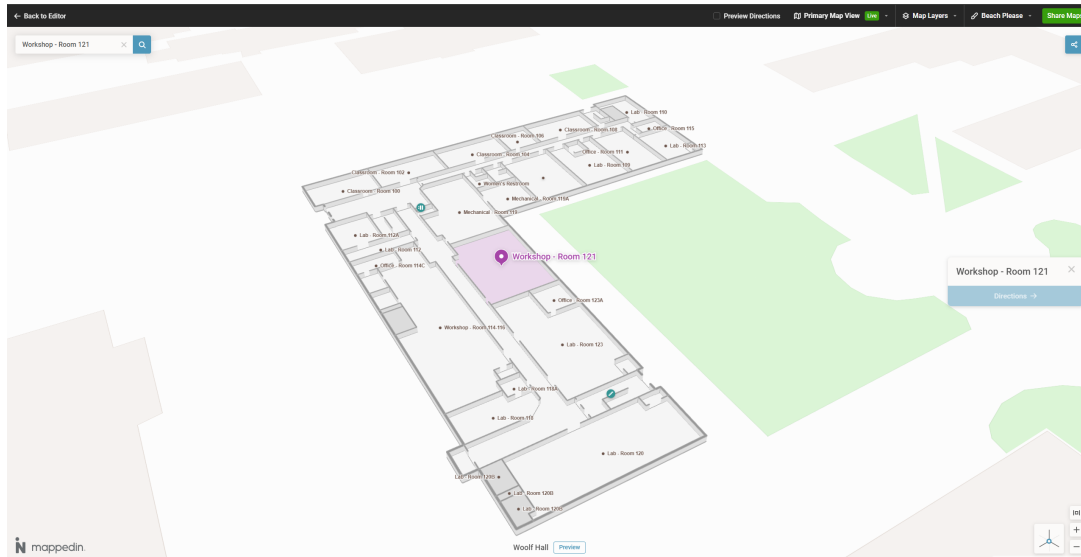


Figure 20: MappedIn Preview

5 External Services & Credentials

The UTA Smart Kiosk relies on several third-party services for hosting, database management, and version control. For ease of maintenance, all administrative accounts for these services are centralized under a single Google account.

Universal Login Credentials

- **Email:** utasmartkiosk@gmail.com (Gmail)
- **Password:** KioskGuys1!

Associated Services The credentials above grant administrative access to the following platforms. Note that they all utilize the exact same email and password combination listed above:

- **Render:** Used for hosting the remote website and backend web services.
- **TiDB:** The cloud database provider used for managing the kiosk's remote MySQL-compatible database schemas.
- **GitHub:** The version control repository where the Smart Kiosk source code is hosted.

6 Contact

If there are any issues or problems that this document did not help with or clarify, contact me directly at:

miguelcervantes87480@gmail.com